

Automated Waste Manager : Smart Dustbin Monitoring System

**Group No. 19 : Suyash Shirsat, Priyanka Vaidya, Devesh
Shelatkar**

Guide : Dr. Shrishti Bhatia

Thadomal Shahani Engineering College

OVERVIEW

- INTRODUCTION
- BLOCK DIAGRAM
- COMPONENTS
- SIMULATION AND CODE
- REFERENCE PAPERS
- FLOW CHART
- WORKING

SMART WASTE MANAGER

- A Smart Waste Manager is an IoT-based system that automates waste level monitoring and disposal using sensors, a motor, and Blynk for remote control.
- It helps in efficient waste management by reducing manual checking and optimizing collection schedules.
- The system consists of:
 - ✓ Ultrasonic Sensor – Measures waste level
 - ✓ Microcontroller (NODEMCU) – Processes data and controls operations
 - ✓ Motor (Servo/DC Motor) – Automates lid movement or compaction
 - ✓ Blynk App – Provides real-time monitoring and alerts

Objective:

- To create a smart and efficient waste disposal system that helps keep the environment clean while using automation and IoT.

COMPONENTS

1

NodeMCU

2

Servo motors

3

Buzzer

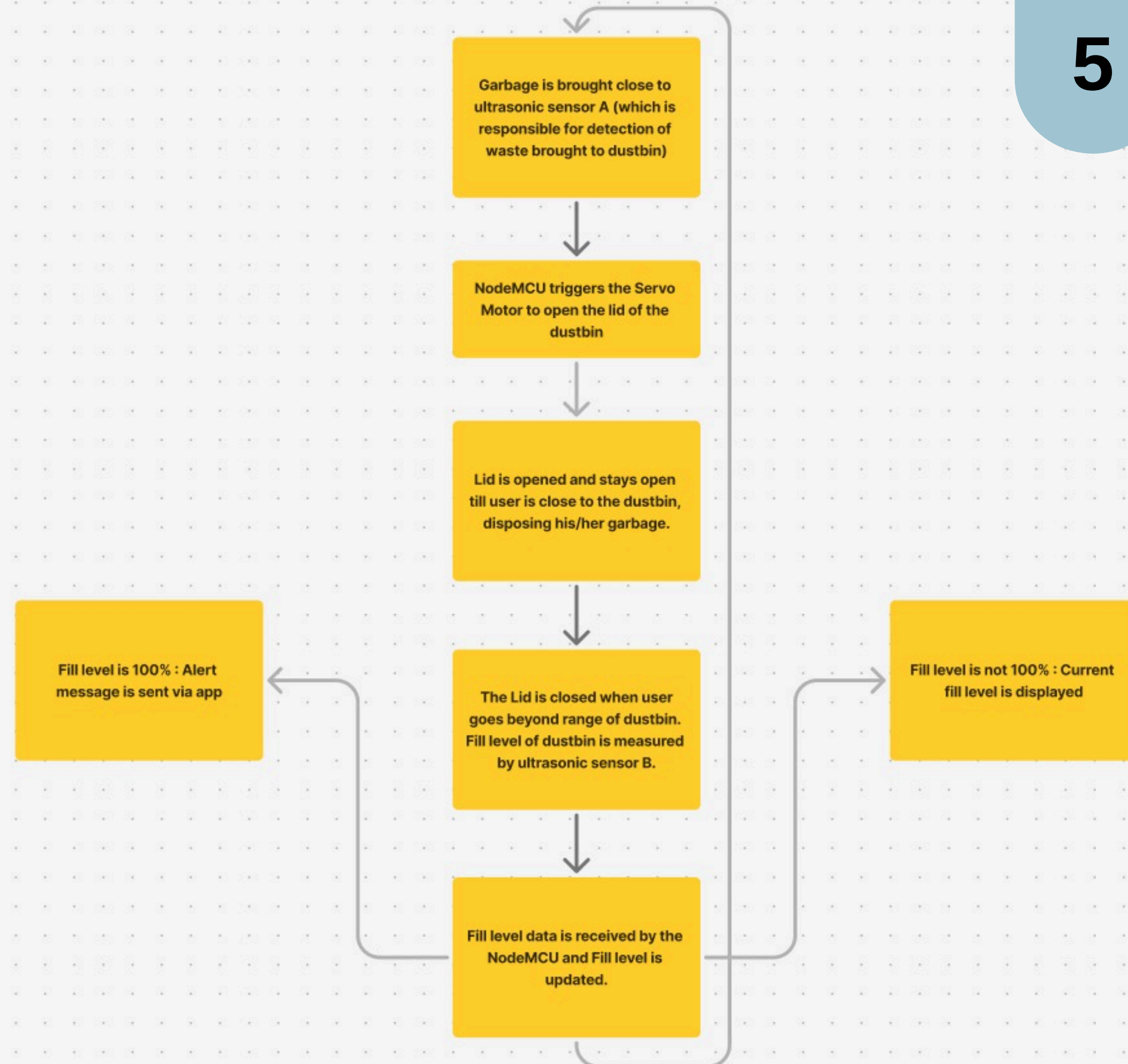
4

Ultrasonic sensors HC-SR04

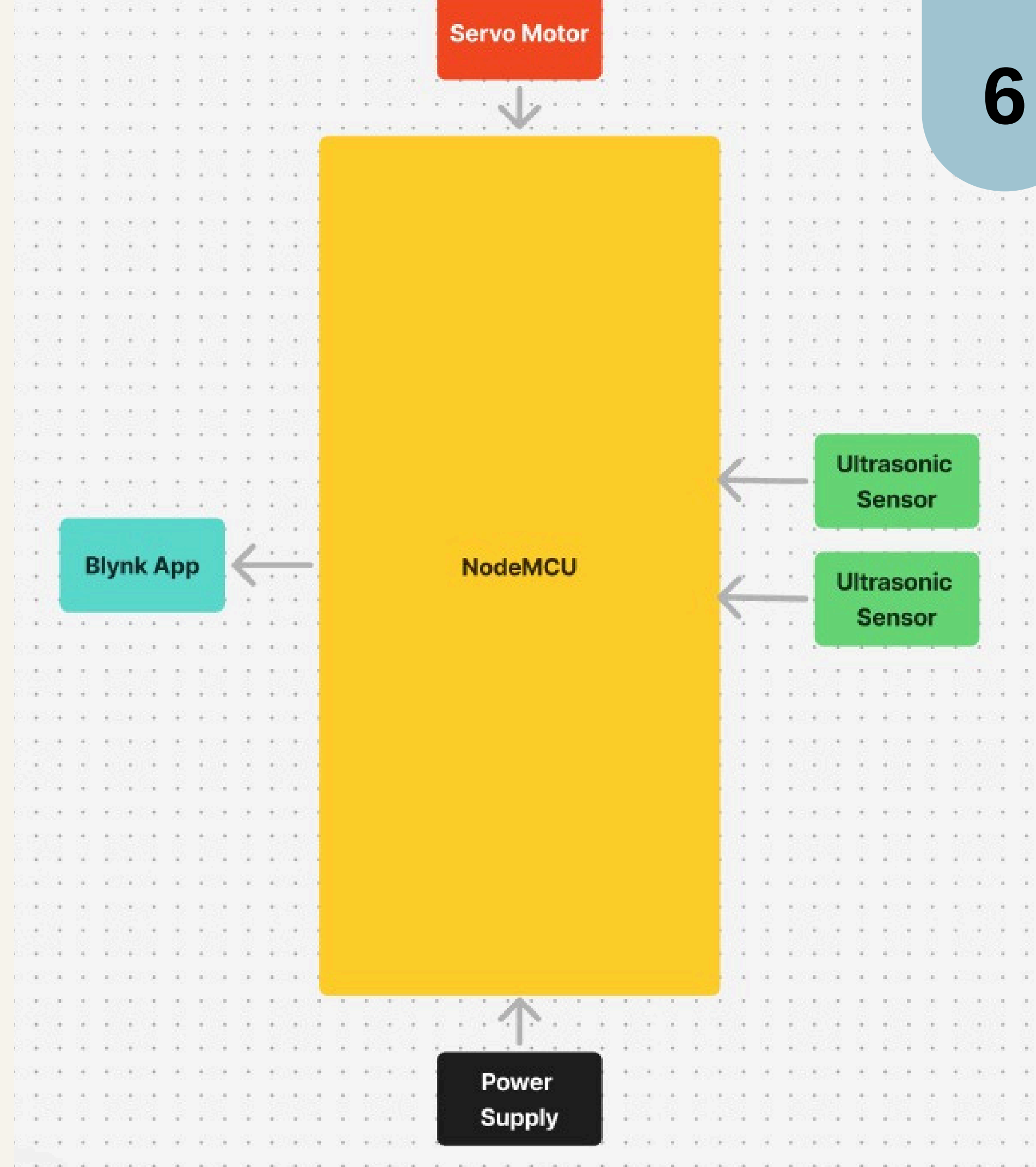
5

Connecting wires

Flowchart



Block Diagram



6

Simulation

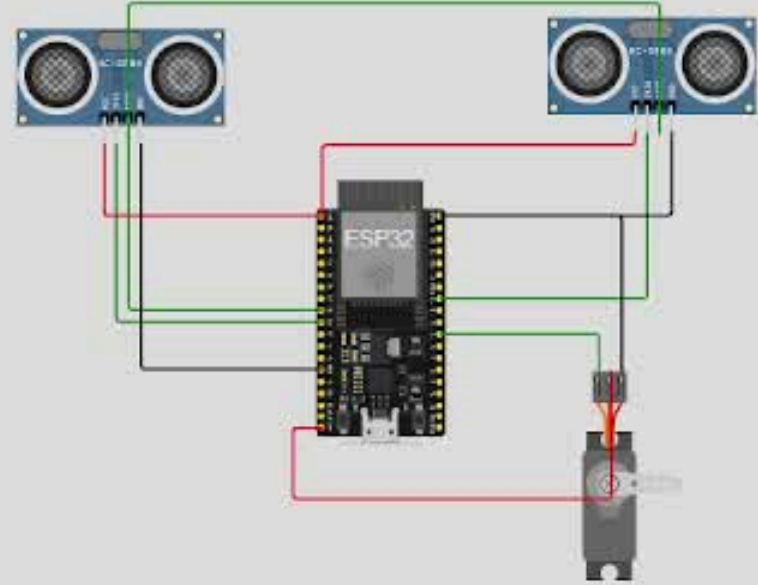
7

WOKWI SAVE SHARE MPR SEM 4 Docs

sketch.ino diagram.json libraries.txt Library Manager

```
1 #include <ESP32Servo.h>
2
3 Servo MyServo;
4
5 #define TRIG_1 19
6 #define TRIG_2 26
7 #define ECHO_PIN 25
8 #define SERVO_PIN1 17
9
10 int alert = 1;
11 int userthreshold = 20;
12 int fillthreshold = 8;
13 bool fillAlertActive = false;
14
15 void setup() {
16     Serial.begin(115200);
17     pinMode(TRIG_1, OUTPUT);
18     pinMode(TRIG_2, OUTPUT);
19     pinMode(ECHO_PIN, INPUT_PULLUP);
20
21     digitalWrite(TRIG_1, LOW);
22     digitalWrite(TRIG_2, LOW);
23     MyServo.attach(SERVO_PIN1);
24     MyServo.write(0);
25 }
26
27 long getDistance(int trigPin) {
```

Simulation



Fill Sensor Distance: 49
Sensor 1 (User Sensor): 50
User Sensor Distance: 50
Sensor 2 (Fill Sensor): 49
Fill Sensor Distance: 49
Sensor 1 (User Sensor): 50
User Sensor Distance: 50

23:31
12-03-2025

Simulation Code:

```
#include <ESP32Servo.h>

Servo MyServo;

#define TRIG_1 19
#define TRIG_2 26
#define ECHO_PIN 25
#define SERVO_PIN1 17

int alert = 1;
int userthreshold = 20;
int fillthreshold = 8;
bool fillAlertActive = false;

void setup() {
    Serial.begin(115200);
    pinMode(TRIG_1, OUTPUT);
    pinMode(TRIG_2, OUTPUT);
    pinMode(ECHO_PIN, INPUT_PULLUP);

    digitalWrite(TRIG_1, LOW);
    digitalWrite(TRIG_2, LOW);
    MyServo.attach(SERVO_PIN1);
    MyServo.write(0);
}

long getDistance(int trigPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 40000);
    if (duration == 0) return -1;

    return duration * 0.034 / 2;
}

void loop() {
    // Check the fill sensor first
    long distance2 = getDistance(TRIG_2);
    Serial.print("Sensor 2 (Fill Sensor): "); Serial.println(distance2);
    checkFillCondition(distance2, fillthreshold);

    // Only check user sensor if bin is not full
    if (!fillAlertActive) {
        long distance1 = getDistance(TRIG_1);
        Serial.print("Sensor 1 (User Sensor): "); Serial.println(distance1);
        checkUserCondition(distance1, userthreshold);
    } else {
        Serial.println("Lid opening disabled as dustbin is filled");
    }

    delay(500);
}

void checkUserCondition(int distance, int threshold) {
    Serial.print("User Sensor Distance: "); Serial.println(distance);

    if (distance > 0 && distance < threshold) {
        openLid();
        while (true) {
            long newDistance = getDistance(TRIG_1);
            if (newDistance >= threshold || newDistance == -1) {
                closeLid();
            }
        }
    }
}
```

```
        break;
    }
    delay(100);
}
}

void checkFillCondition(int distance, int threshold) {
    Serial.print("Fill Sensor Distance: "); Serial.println(distance);

    if (distance == 0) {
        Serial.println("ERROR: No echo received from Fill Sensor!");
    } else if (distance > 0 && distance < threshold) {
        if (!fillAlertActive) {
            Serial.println("Dustbin is almost full.");
            fillAlertActive = true;
        }
    } else {
        if (fillAlertActive) {
            Serial.println("Dustbin has space.");
            fillAlertActive = false;
        }
    }
}

void openLid() {
    Serial.println("Opening lid...");
    MyServo.write(180);
    delay(500);
}

void closeLid() {
    Serial.println("Closing lid...");
    MyServo.write(0);
    delay(500);
}
```


WORKING

Smart Dustbin – Working Overview

The Smart Dustbin operates using ultrasonic sensors and a servo motor to enable automatic, touch-free waste disposal. It enhances hygiene by detecting objects near the bin and opening the lid automatically.

Object Detection & Lid Operation:

An ultrasonic sensor is placed at the front of the dustbin to detect when a person or object (such as waste) comes close. If an object is detected within 20 cm, the servo motor rotates to open the lid, allowing waste to be disposed of.

Waste Level Monitoring:

A second ultrasonic sensor inside the bin measures the fill level by calculating the distance between the sensor and the waste surface.

As waste accumulates, the sensor monitors the decreasing distance to determine how full the bin is.

User Alerts & Smart Features:

When the dustbin reaches its maximum capacity, the system can trigger alerts (such as a buzzer, LED, or notification via Wi-Fi).

The system prevents the lid from opening further when the bin is full, ensuring no overflow.

Power & Operation:

The system runs on a NodeMCU (ESP8266) microcontroller, which controls the sensors and servo motor.

The ESP8266 can be further integrated with Wi-Fi to send notifications or updates to a mobile app or cloud-based dashboard for remote monitoring.

WORKING PRINCIPLE

10

Microcontroller & Peripheral Control:

- The NodeMCU ESP8266 acts as the brain of the system, managing two HC-SR04 ultrasonic sensors and a servo motor.
- It reads data from the sensors to detect when a user is nearby and monitors the bin's fill level.
- Based on these readings, it decides whether to open or close the dustbin lid.

NodeMCU:

- The NodeMCU ESP8266 is a microcontroller with built-in Wi-Fi capabilities.
- It processes data from the ultrasonic sensors and controls the servo motor based on logical conditions.
- It communicates with peripherals using GPIO pins and operates at 3.3V logic levels.
- The PWM (Pulse Width Modulation) output is used to control the position of the servo motor.
- It also supports serial communication, enabling real-time monitoring of sensor values and system status.
- Sensor Operation & Distance Measurement:
- The dustbin has two ultrasonic sensors:
- User Sensor (TRIG_1, ECHO_PIN): Detects people approaching the bin.
- Fill Sensor (TRIG_2, ECHO_PIN): Checks how full the bin is.
- The NodeMCU sends a brief 10μs pulse to the sensor's trigger pin.
- The sensor then sends out an ultrasonic wave, which reflects back when it hits an object.
- The Echo pin stays HIGH while the wave returns, and the system calculates distance using:
- $\text{Distance} = (\text{Speed of Sound} \times \text{Time}) \div 2$
- If no signal is received, an error message is displayed.

WORKING PRINCIPLE

11

Ultrasonic Sensor:

- The HC-SR04 ultrasonic sensor consists of a transmitter and a receiver.
- The transmitter emits an ultrasonic sound pulse (40 kHz frequency), which bounces off objects.
- The receiver detects the reflected wave and converts it into an electrical signal.
- The time taken for the wave to return helps calculate the distance to the object.
- Decision Making & Servo Control:
- Checking the Fill Level:
- If the bin is not full, the system allows the lid to open.
- If the bin is full, a Fill Alert is activated, preventing the lid from opening.
- Detecting a User:
- If someone comes within 20 cm, the lid opens automatically.
- The system keeps checking the distance in real-time.
- If the person moves away or no object is detected, the lid closes again.
- How the Servo Motor Works:
- Controlled by PWM (Pulse Width Modulation) signals from the ESP8266.
- Rotates 180° to open the lid and returns to 5° to close it.

WORKING PRINCIPLE

12

Servo Motor:

- The servo motor operates based on PWM signals, where the pulse width determines the position.
- A pulse width of $\sim 500\mu\text{s}$ moves the servo to 5° , and $\sim 2500\mu\text{s}$ moves it to 180° .
- The servo has an internal feedback mechanism that ensures precise angular positioning.
- The NodeMCU sends variable pulse signals to adjust the servo's movement as required.
- Real-time Monitoring & Serial Communication:
 - The ESP8266 continuously sends status updates to the Serial Monitor.
 - Some example messages include:
 - Sensor readings to help with debugging.
 - "Opening lid..." when a user is detected.
 - "Lid opening disabled due to fill alert!" if the bin is full.
- Power & Connectivity Considerations:
 - The NodeMCU operates on 3.3V, while the sensors and servo require 5V.
 - To ensure smooth operation, an external power source may be needed for the servo motor.

- The NodeMCU manages smart dustbin automation, detecting user presence and waste levels.
- It ensures hands-free operation by automatically opening and closing the lid based on proximity sensing.
- Could be expanded with Wi-Fi integration for remote monitoring or a mobile notification system.

Research Papers

14

<https://sci-hub.se/https://ieeexplore.ieee.org/document/8529600>

<https://sci-hub.se/https://ieeexplore.ieee.org/document/8537245>

<https://sci-hub.se/https://ieeexplore.ieee.org/document/8519845>

CONCLUSION

This automated, touch-free smart dustbin helps improve hygiene, reduce manual intervention, and prevent overflow by providing real-time waste level monitoring. It is ideal for homes, offices, and public places looking for an efficient waste management solution

THANK YOU